Smart ATM Cash Refill Prediction System

Machine Learning Approach for Predicting ATM Cash Refill Requirements

MACHINE LEARNING PROJECT

Python	**Pandas**	**NumPy**
Matplotlib	**Seaborn**	**Scikit-learn**

Prepared by: [Your Name]

Portfolio-ready documentation based on the supplied Jupyter Notebook.

2. Project Overview

This project builds a machine learning classification system for predicting whether an ATM needs a cash refill. The supplied notebook uses historical ATM-related operational, transaction, location, calendar, and environmental variables and trains multiple classification algorithms.

The prediction target is **Needs_Refill**. The notebook encodes categorical variables, separates the target from the features, trains four classification models, compares their test accuracy, evaluates a Random Forest model with a confusion matrix and classification report, examines feature importance, tunes Random Forest with GridSearchCV, and saves the selected model using Joblib.

The business motivation is straightforward: an ATM should have enough cash to meet customer demand without creating avoidable refill activity. A prediction system can support refill planning by flagging ATMs that are likely to require replenishment.

Source boundary: All measured dataset values, model scores, feature-importance values, and prediction outputs in this document are taken from the provided notebook. Where a point is a proposed application or improvement, it is explicitly labeled as such.

3. Business Problem

ATMs need sufficient cash to serve withdrawals. If cash is depleted, customers may be unable to withdraw money. On the other hand, unnecessary refilling can create avoidable operational and transportation effort.

Objective: use historical ATM-related data to predict whether an ATM **Needs_Refill**.

Input → Machine Learning Model → Refill Prediction

Historical ATM Data
↓
Data Preparation
↓
Classification Model
↓
Needs_Refill Prediction

4. Project Objectives

• Analyze ATM transaction and operational data.

• Identify patterns associated with refill requirements through exploratory analysis.

• Prepare categorical and numerical data for machine learning.

• Train and compare Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting.

• Evaluate the Random Forest model with a confusion matrix, classification report, and feature importance.

• Tune Random Forest hyperparameters using GridSearchCV.

• Save the selected trained model with Joblib for reuse.

5. Dataset Overview

The notebook reports **10,000 rows and 25 columns**. The dataset contains 15 object columns and 10 numerical columns before encoding. All columns report 10,000 non-null values, and duplicate analysis returns zero duplicates.

The target variable is **Needs_Refill**. Before encoding, it is categorical with two unique values; the notebook's first five rows show values such as Yes and No.

Item	Notebook result
Rows	10,000
Columns	25
Object columns	15
Numerical columns	10
Missing values	0 across all 25 columns
Duplicate records	0
Target	Needs_Refill

Feature inventory

Feature	Type	Notebook-based description
ATM_ID	object → encoded	ATM identifier
Date	object → encoded	Date recorded for the ATM observation
Day_of_Week	object → encoded	Day of week
Month	object → encoded	Month
Location_Type	object → encoded	Location category
ATM_Type	object → encoded	ATM type
Nearby_Market	object → encoded	Whether a nearby market is recorded
Festival_Season	object → encoded	Festival-season indicator
Salary_Week	object → encoded	Salary-week indicator
Holiday	object → encoded	Holiday indicator
Weather	object → encoded	Weather category
Temperature_C	float64	Temperature in Celsius
Previous_Day_Withdrawals	int64	Previous-day withdrawal measure
Average_Withdrawal_Amount	float64	Average withdrawal amount
Cash_Loaded	int64	Cash loaded into ATM
Current_Cash_Balance	int64	Current cash balance
Transactions_Today	int64	Today's transaction count
Working_Hours	int64	Working hours
ATM_Age_Years	int64	ATM age in years
Maintenance_Due	object → encoded	Maintenance-due indicator
Nearest_Bank_Distance_km	float64	Distance to nearest bank in km
Population_Density	object → encoded	Population-density category
Nearby_Businesses	int64	Nearby business count
Weekend	object → encoded	Weekend indicator
Needs_Refill	object → encoded	Classification target

Feature descriptions above are conservative interpretations of the supplied column names and notebook values; no unsupported business definition has been added.

6. Technology Stack

Tool / Library	Purpose in the notebook
Python	Programming and model workflow
Pandas	Data loading, inspection, transformation, and tabular analysis
NumPy	Numerical operations / array outputs
Matplotlib	Plotting
Seaborn	Statistical visualizations
Scikit-learn	Preprocessing, model training, evaluation, and tuning
Joblib	Saving and loading the trained model

7. Importing Libraries

The notebook imports the libraries required for data manipulation, visualization, preprocessing, classification, model evaluation, tuning, and model persistence.

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

import joblib

import warnings
warnings.filterwarnings("ignore")
```

8. Loading the Dataset

The original notebook loads a CSV from a local Windows development path. For portfolio presentation, the path is represented below with a cleaner generic filename while preserving the same CSV-loading methodology.

```
df = pd.read_csv("smart_atm_cash_refill_dataset.csv")
```

Original notebook path: `C:/Users/sahus/Downloads/smart_atm_cash_refill_dataset.csv`. This is a local development path and is not presented as a deployment path.

9. Initial Data Exploration

9.1 First Few Records

```
df.head()
```

```
ATM_ID        Date Day_of_Week     Month Location_Type ATM_Type  \
0  ATM1327  2023-04-25     Tuesday     April         Urban  Offsite
1  ATM1125  2023-08-17    Thursday    August         Urban   Onsite
2  ATM1346  2024-07-12      Friday      July         Rural  Offsite
3  ATM1216  2023-02-02    Thursday  February         Rural  Offsite
4  ATM1111  2023-08-27      Sunday    August         Urban  Offsite

  Nearby_Market Festival_Season Salary_Week Holiday  ... Current_Cash_Balance  \
0            No              No          No      No  ...                10000
1            No             Yes          No      No  ...               738699
2           Yes             Yes          No      No  ...              1813354
3            No              No         Yes     Yes  ...               448529
4            No              No          No     Yes  ...                10000

   Transactions_Today  Working_Hours  ATM_Age_Years  Maintenance_Due  \
0                 561             16              9               No
1                 426             16              4              Yes
2                 142             16             19               No
3                 122             16              3               No
4                 445             24             20               No

   Nearest_Bank_Distance_km  Population_Density  Nearby_Businesses  Weekend  \
0                     13.21                High                 88       No
1                     23.73                High                 66       No
2                     19.42                 Low                 24       No
3                     20.09                 Low                  0       No
4                      8.26                High                 52      Yes

  Needs_Refill
0          Yes
1          Yes
2           No
3           No
4           No

[5 rows x 25 columns]
```

9.2 Dataset Shape

```
df.shape
```

```
(10000, 25)
```

9.3 Dataset Information

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 25 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   ATM_ID                    10000 non-null  object
 1   Date                      10000 non-null  object
 2   Day_of_Week               10000 non-null  object
 3   Month                     10000 non-null  object
 4   Location_Type             10000 non-null  object
 5   ATM_Type                  10000 non-null  object
 6   Nearby_Market             10000 non-null  object
 7   Festival_Season           10000 non-null  object
 8   Salary_Week               10000 non-null  object
 9   Holiday                   10000 non-null  object
 10  Weather                   10000 non-null  object
 11  Temperature_C             10000 non-null  float64
 12  Previous_Day_Withdrawals  10000 non-null  int64
 13  Average_Withdrawal_Amount 10000 non-null  float64
 14  Cash_Loaded               10000 non-null  int64
 15  Current_Cash_Balance      10000 non-null  int64
 16  Transactions_Today        10000 non-null  int64
 17  Working_Hours             10000 non-null  int64
 18  ATM_Age_Years             10000 non-null  int64
 19  Maintenance_Due           10000 non-null  object
 20  Nearest_Bank_Distance_km  10000 non-null  float64
 21  Population_Density        10000 non-null  object
 22  Nearby_Businesses         10000 non-null  int64
 23  Weekend                   10000 non-null  object
```

```
 24  Needs_Refill              10000 non-null  object
dtypes: float64(3), int64(7), object(15)
memory usage: 1.9+ MB
```

9.4 Descriptive Statistics — Numerical

```
df.describe()
```

```
Temperature_C  Previous_Day_Withdrawals  Average_Withdrawal_Amount  \
count   10000.000000              10000.000000               10000.000000
mean       31.396010                405.838100                4530.521600
std         7.745268                174.778797                1934.263443
min        18.000000                 20.000000                 500.000000
25%        24.800000                274.000000                3183.032500
50%        31.200000                409.000000                4527.145000
75%        38.000000                540.000000                5859.035000
max        45.000000                900.000000               10000.000000

        Cash_Loaded  Current_Cash_Balance  Transactions_Today  Working_Hours  \
count  1.000000e+04          1.000000e+04        10000.000000   10000.000000
mean   1.744650e+06          5.851436e+05          406.389500      19.346400
std    7.195728e+05          7.162498e+05          164.582874       3.406518
min    5.013970e+05          1.000000e+04           20.000000      16.000000
25%    1.119358e+06          1.000000e+04          282.000000      16.000000
50%    1.739031e+06          2.146080e+05          413.000000      18.000000
75%    2.373264e+06          1.046187e+06          540.000000      24.000000
max    2.999930e+06          2.945306e+06          869.000000      24.000000

       ATM_Age_Years  Nearest_Bank_Distance_km  Nearby_Businesses
count   10000.000000              10000.000000       10000.000000
mean       10.533600                 12.654676          44.094100
std         5.807172                  7.212740          24.822765
min         1.000000                  0.200000           0.000000
25%         5.000000                  6.300000          24.000000
50%        11.000000                 12.870000          44.500000
75%        16.000000                 18.970000          64.000000
max        20.000000                 25.000000         100.000000
```

9.4 Descriptive Statistics — Categorical

```
df.describe(include="object")
```

```
ATM_ID        Date Day_of_Week   Month Location_Type ATM_Type  \
count     10000       10000       10000   10000         10000    10000
unique      500         731           7      12             3        2
top     ATM1309  2024-10-08    Thursday  August         Urban   Onsite
freq         33          25        1456     874          4502     6020

       Nearby_Market Festival_Season Salary_Week Holiday Weather  \
count          10000           10000       10000   10000   10000
unique             2               2           2       2       3
top              Yes              No          No      No   Sunny
freq            5033            8500        7264    9029    5503

       Maintenance_Due Population_Density Weekend Needs_Refill
count            10000              10000   10000        10000
unique               2                  3       2            2
top                 No               High      No          Yes
freq              8801               4095    7202         5000
```

9.5 Missing Values

```
df.isnull().sum()
```

```
ATM_ID                      0
Date                        0
Day_of_Week                 0
Month                       0
Location_Type               0
ATM_Type                    0
Nearby_Market               0
Festival_Season             0
Salary_Week                 0
Holiday                     0
Weather                     0
Temperature_C               0
Previous_Day_Withdrawals    0
Average_Withdrawal_Amount   0
Cash_Loaded                 0
Current_Cash_Balance        0
Transactions_Today          0
```

```
Working_Hours               0
ATM_Age_Years               0
Maintenance_Due             0
Nearest_Bank_Distance_km    0
Population_Density          0
Nearby_Businesses           0
Weekend                     0
Needs_Refill                0
dtype: int64
```

9.6 Duplicate Records

```
df.duplicated().sum()
```

```
np.int64(0)
```

9.7 Data Types

```
df.dtypes
```

```
ATM_ID                       object
Date                         object
Day_of_Week                  object
Month                        object
Location_Type                object
ATM_Type                     object
Nearby_Market                object
Festival_Season              object
Salary_Week                  object
Holiday                      object
Weather                      object
Temperature_C               float64
Previous_Day_Withdrawals      int64
Average_Withdrawal_Amount   float64
Cash_Loaded                   int64
Current_Cash_Balance          int64
Transactions_Today            int64
Working_Hours                 int64
ATM_Age_Years                 int64
Maintenance_Due              object
Nearest_Bank_Distance_km    float64
Population_Density           object
Nearby_Businesses             int64
Weekend                      object
Needs_Refill                 object
dtype: object
```

9.8 Column Names

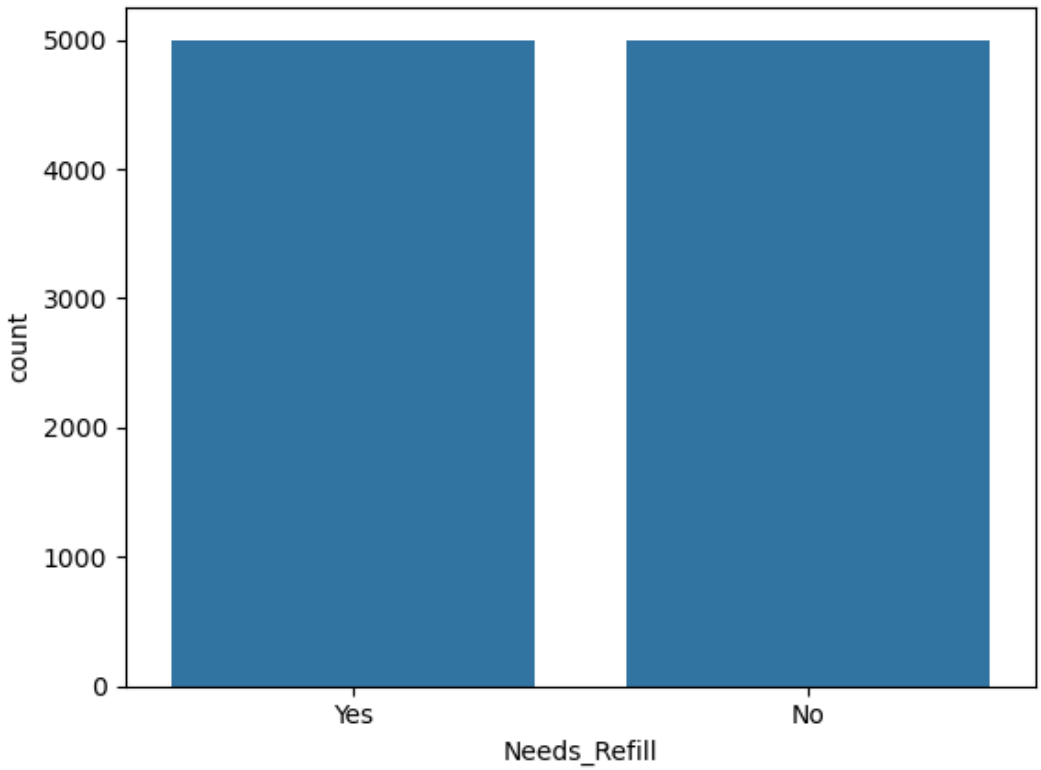
```
df.columns
```

```
Index(['ATM_ID', 'Date', 'Day_of_Week', 'Month', 'Location_Type', 'ATM_Type',
       'Nearby_Market', 'Festival_Season', 'Salary_Week', 'Holiday', 'Weather',
       'Temperature_C', 'Previous_Day_Withdrawals',
       'Average_Withdrawal_Amount', 'Cash_Loaded', 'Current_Cash_Balance',
       'Transactions_Today', 'Working_Hours', 'ATM_Age_Years',
       'Maintenance_Due', 'Nearest_Bank_Distance_km', 'Population_Density',
       'Nearby_Businesses', 'Weekend', 'Needs_Refill'],
      dtype='object')
```

10. Exploratory Data Analysis

Target Variable Distribution

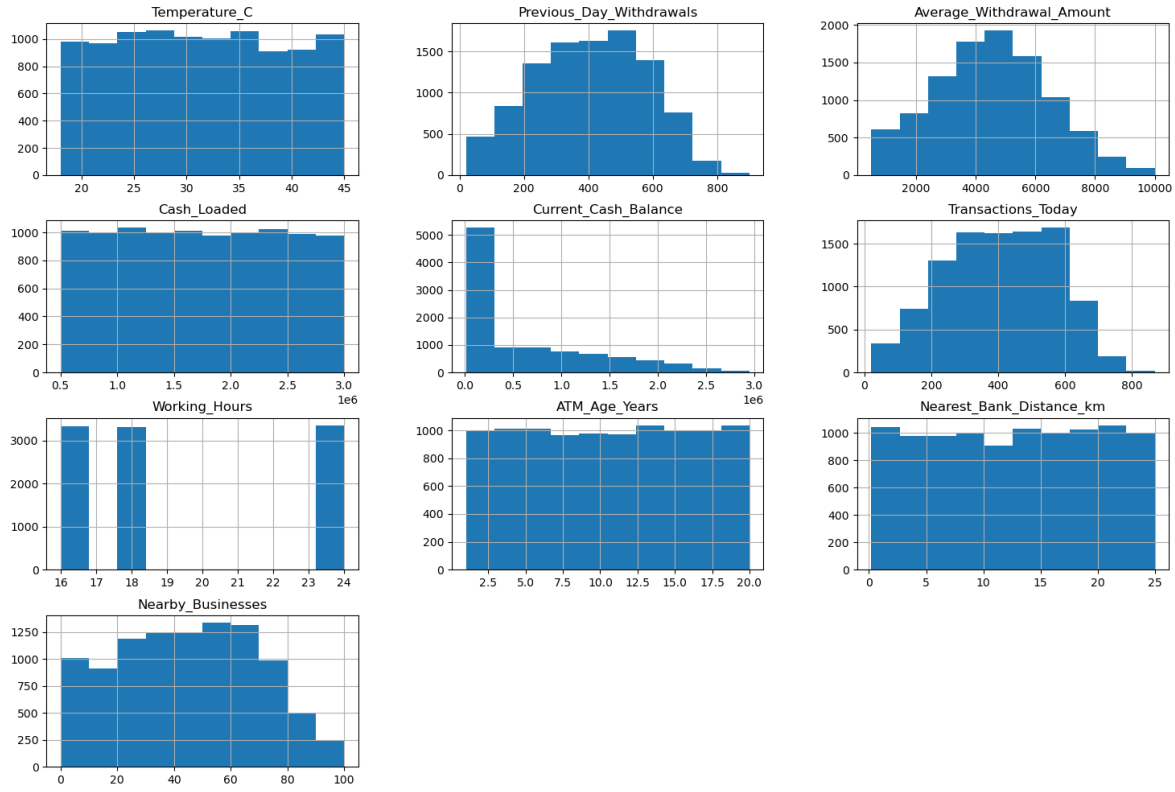
The notebook uses a count plot for Needs_Refill. The categorical target has two values, and the categorical summary reports 5,000 observations for the most frequent value Yes.



Original visualization output from the supplied notebook.

Numerical Feature Distribution

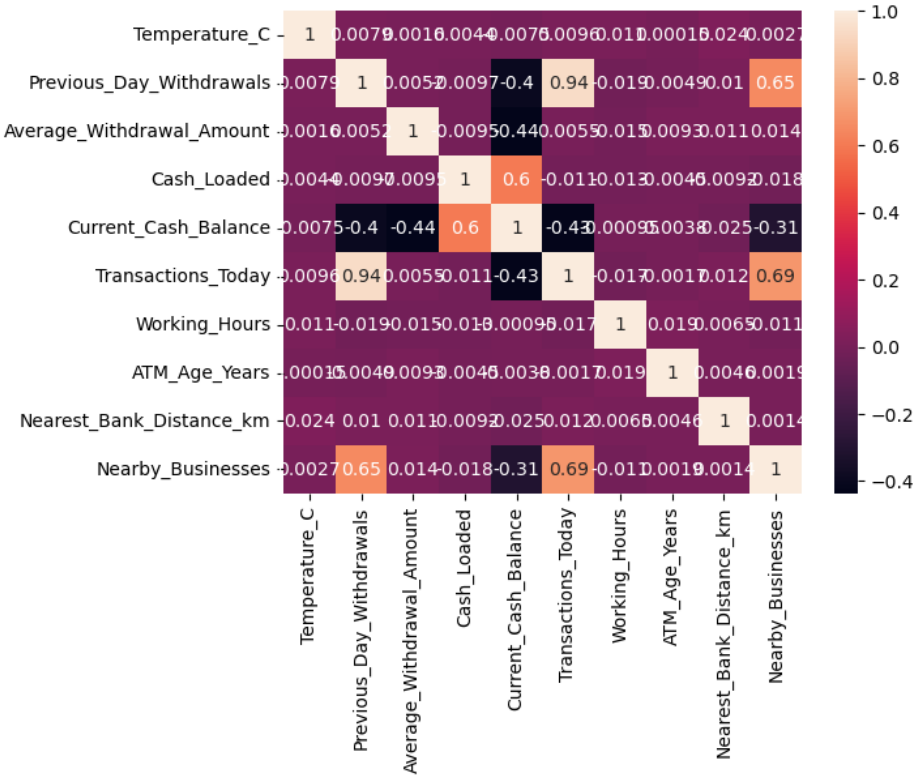
The notebook generates histograms for the numerical variables to inspect their distributions.



Original visualization output from the supplied notebook.

Correlation Analysis

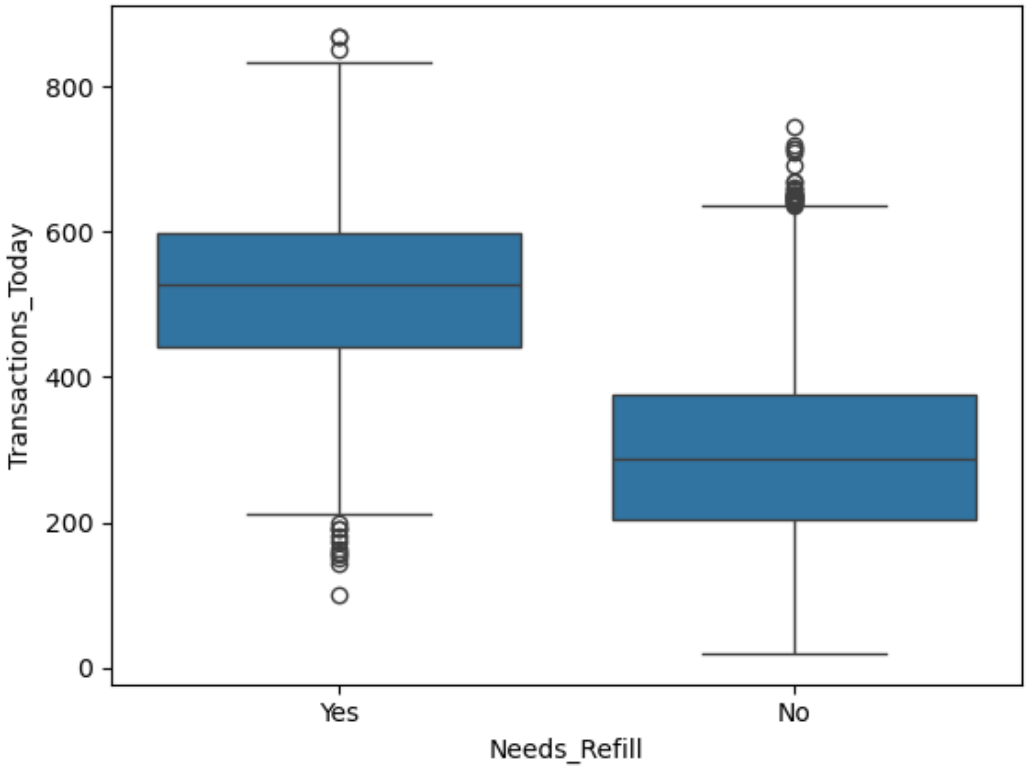
A correlation heatmap is generated for numerical variables using pandas numeric correlation.



Original visualization output from the supplied notebook.

Transactions Today by Refill Target

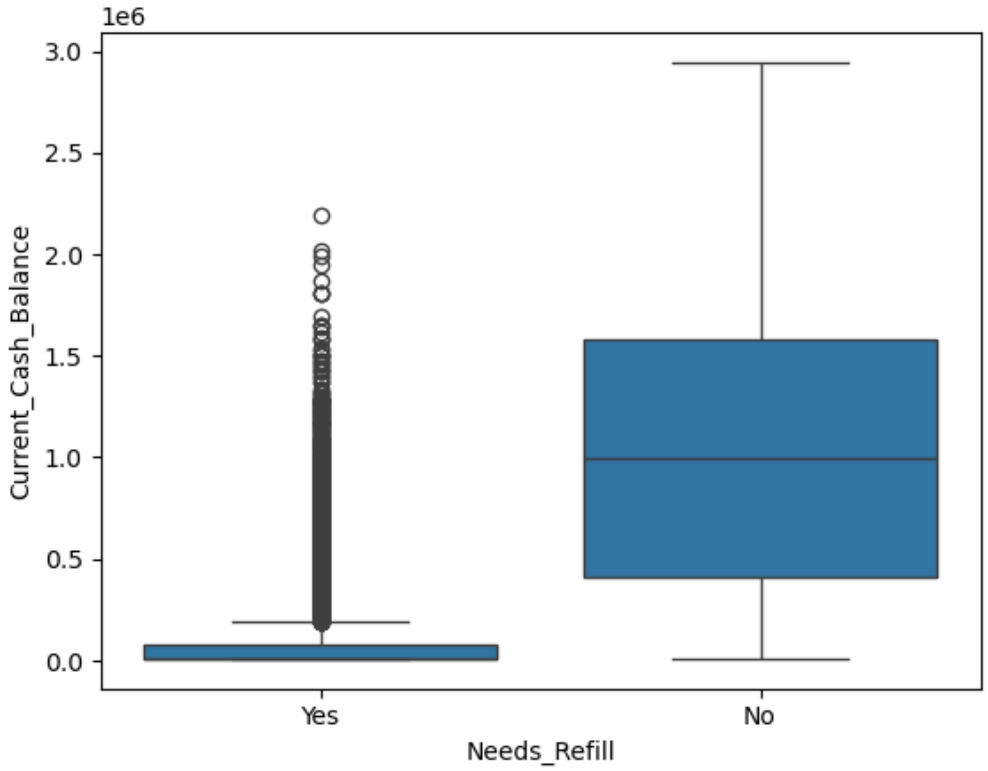
A box plot compares Transactions_Today across the Needs_Refill classes.



Original visualization output from the supplied notebook.

Current Cash Balance by Refill Target

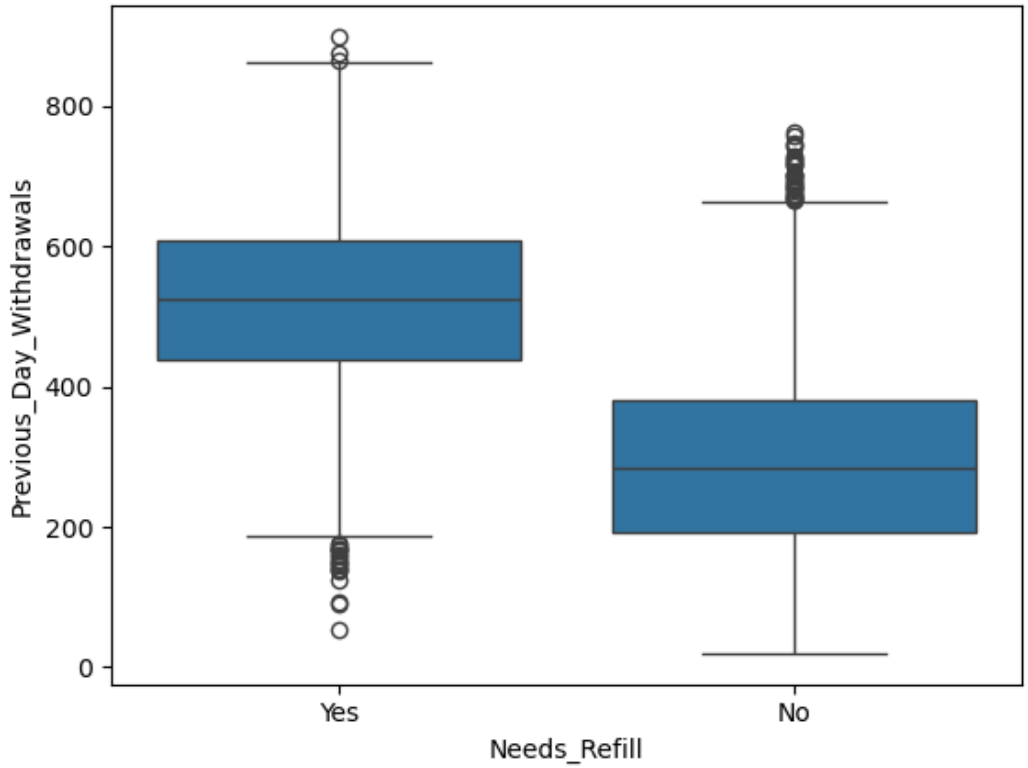
A box plot compares Current_Cash_Balance across the Needs_Refill classes.



Original visualization output from the supplied notebook.

Previous Day Withdrawals by Refill Target

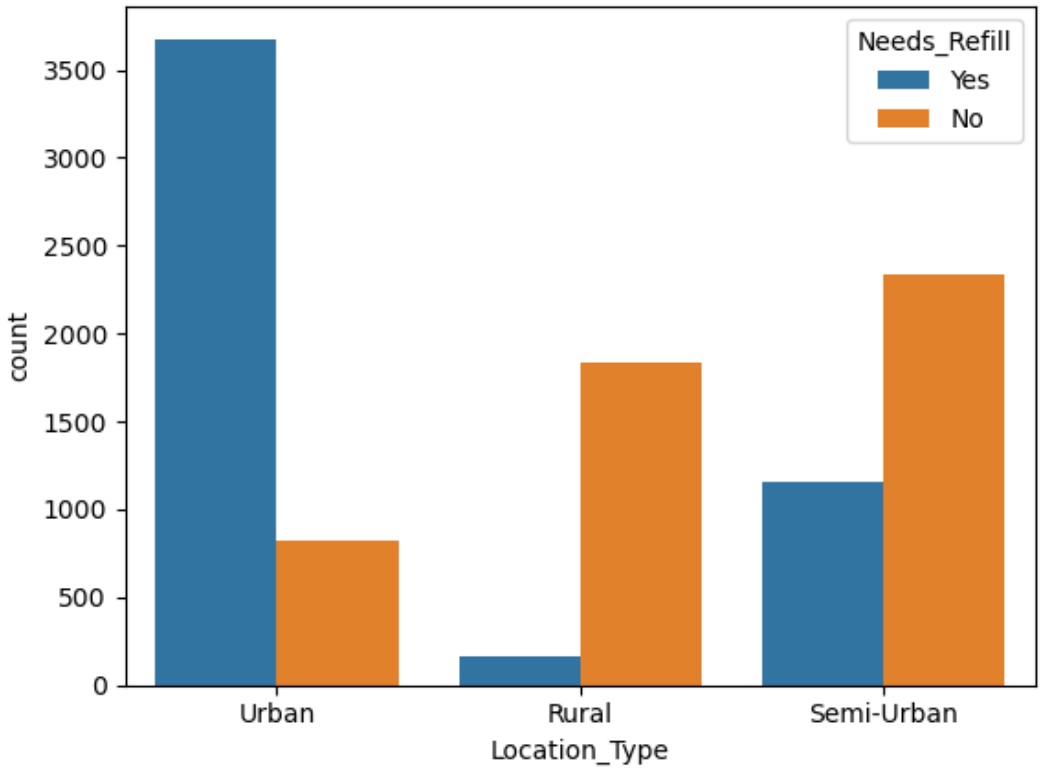
A box plot compares Previous_Day_Withdrawals across the Needs_Refill classes.



Original visualization output from the supplied notebook.

Location Type and Refill Target

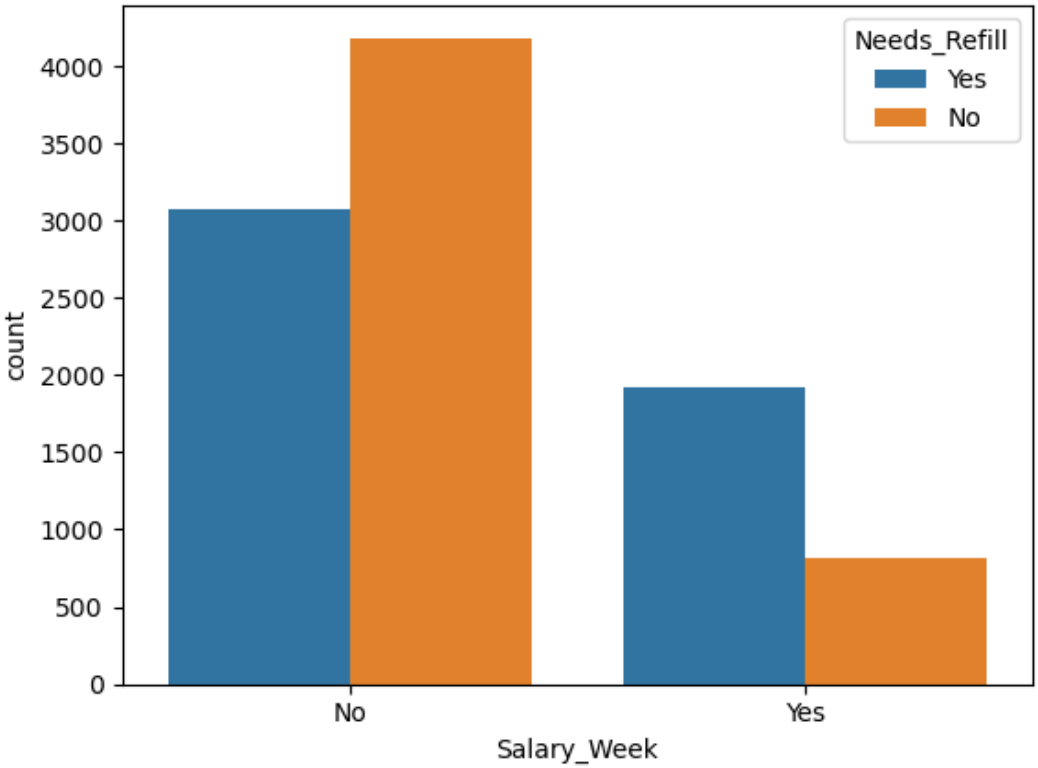
A count plot compares Needs_Refill across Location_Type categories.



Original visualization output from the supplied notebook.

Salary Week and Refill Target

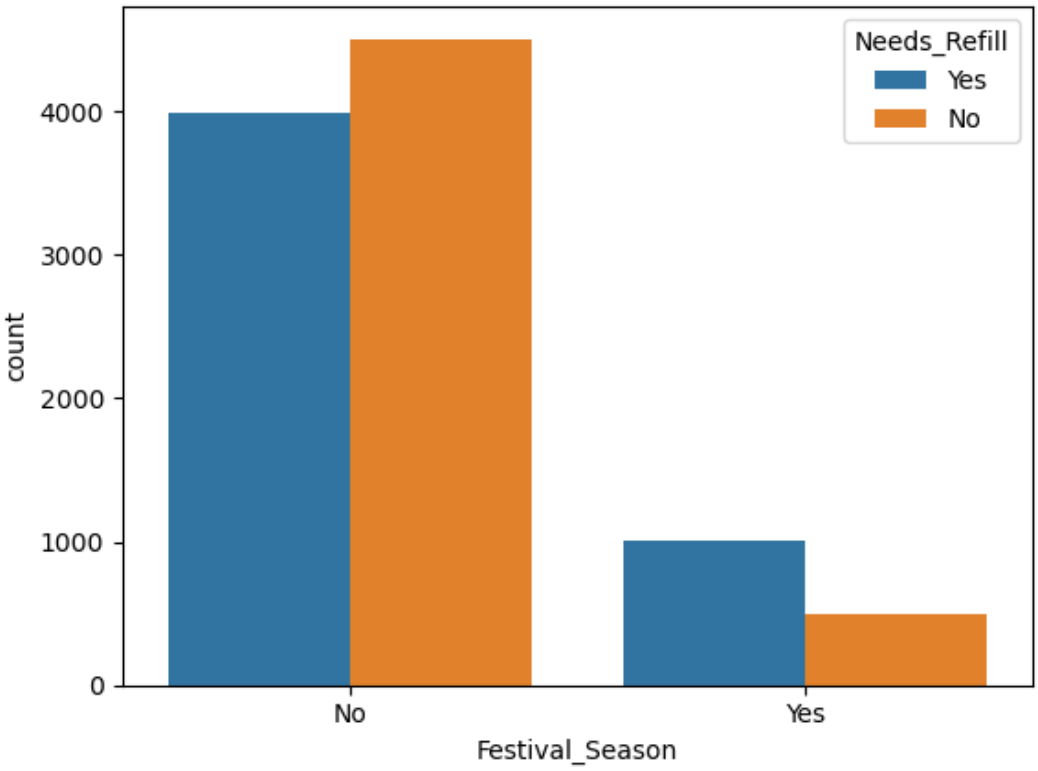
A count plot compares Needs_Refill across Salary_Week categories.



Original visualization output from the supplied notebook.

Festival Season and Refill Target

A count plot compares Needs_Refill across Festival_Season categories.



Original visualization output from the supplied notebook.

EDA note: the notebook provides visual outputs, but it does not include written statistical conclusions for each chart. Therefore, this PDF avoids inventing chart-specific findings beyond what is directly represented by the notebook outputs and summary tables.

11. Data Preprocessing

The notebook identifies categorical and numerical columns, then applies **LabelEncoder** to every categorical column. This includes the target column, Needs_Refill. After encoding, all model inputs are numeric.

The feature matrix X is created by dropping Needs_Refill, while y contains Needs_Refill. The notebook then performs an 80/20 train-test split with random_state=42.

Categorical and numerical column identification

```
categorical_cols = df.select_dtypes(include="object").columns
numerical_cols = df.select_dtypes(exclude="object").columns

print("Categorical Columns:")
print(categorical_cols)

print("\nNumerical Columns:")
print(numerical_cols)
```

Categorical encoding

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

for col in categorical_cols:
    df[col] = le.fit_transform(df[col])

df.head()
```

Feature and target definition

```
X = df.drop("Needs_Refill", axis=1)
y = df["Needs_Refill"]
```

The resulting feature matrix contains 10,000 rows and 24 features; the target contains 10,000 values.

```
print("Features Shape:", X.shape)
print("Target Shape:", y.shape)
```

12. Feature and Target Definition

Target variable: Needs_Refill

X represents the 24 input features remaining after removing Needs_Refill.

y represents the encoded refill target.

This is a classification problem because the model predicts a class label rather than a continuous numerical value.

```
X = df.drop("Needs_Refill", axis=1)
y = df["Needs_Refill"]
```

13. Train-Test Split

The notebook uses test_size=0.2, meaning 20% of the 10,000 observations are reserved for testing and 80% are used for training. random_state=42 makes the split reproducible.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
```

```
    test_size=0.2,
    random_state=42
)

X_train : (8000, 24)
X_test  : (2000, 24)
y_train : (8000,)
y_test  : (2000,)
```

14. Machine Learning Models

Model 1 — Logistic Regression

A linear classification baseline. The notebook uses max_iter=1000, fits the model on the training data, predicts the test set, and records an accuracy of 0.8795.

```
from sklearn.linear_model import LogisticRegression

lr_model = LogisticRegression(max_iter=1000)

lr_model.fit(X_train, y_train)

lr_pred = lr_model.predict(X_test)

from sklearn.metrics import accuracy_score

lr_accuracy = accuracy_score(y_test, lr_pred)

print("Logistic Regression Accuracy:", lr_accuracy)

Logistic Regression Accuracy: 0.8795
```

Model 2 — Decision Tree

A tree-based classifier that can represent non-linear decision rules. The notebook uses random_state=42 and records a test accuracy of 0.9295.

```
from sklearn.tree import DecisionTreeClassifier

dt_model = DecisionTreeClassifier(random_state=42)

dt_model.fit(X_train, y_train)

dt_pred = dt_model.predict(X_test)

dt_accuracy = accuracy_score(y_test, dt_pred)

print("Decision Tree Accuracy:", dt_accuracy)

Decision Tree Accuracy: 0.9295
```

Model 3 — Random Forest

An ensemble of decision trees. The notebook uses n_estimators=100 and random_state=42 and records a test accuracy of 0.9415.

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train, y_train)

rf_pred = rf_model.predict(X_test)

rf_accuracy = accuracy_score(y_test, rf_pred)

print("Random Forest Accuracy:", rf_accuracy)

Random Forest Accuracy: 0.9415
```

Model 4 — Gradient Boosting

A boosting-based classifier. The notebook records a test accuracy of 0.9645, the highest among the four models evaluated directly on the test set.

```
from sklearn.ensemble import GradientBoostingClassifier

gb_model = GradientBoostingClassifier(random_state=42)

gb_model.fit(X_train, y_train)

gb_pred = gb_model.predict(X_test)

gb_accuracy = accuracy_score(y_test, gb_pred)

print("Gradient Boosting Accuracy:", gb_accuracy)

Gradient Boosting Accuracy: 0.9645
```

15. Model Comparison

The notebook creates model_results and sorts the models by Accuracy in descending order. The measured test-set results are:

Model	Accuracy
Gradient Boosting	0.9645 (96.45%)
Random Forest	0.9415 (94.15%)
Decision Tree	0.9295 (92.95%)
Logistic Regression	0.8795 (87.95%)

Best direct test-set model: Gradient Boosting, with accuracy 0.9645 (96.45%) in the supplied notebook.

16. Model Evaluation

The notebook evaluates the Random Forest model with a confusion matrix and classification report. This is separate from the direct model comparison, where Gradient Boosting has the highest test accuracy.

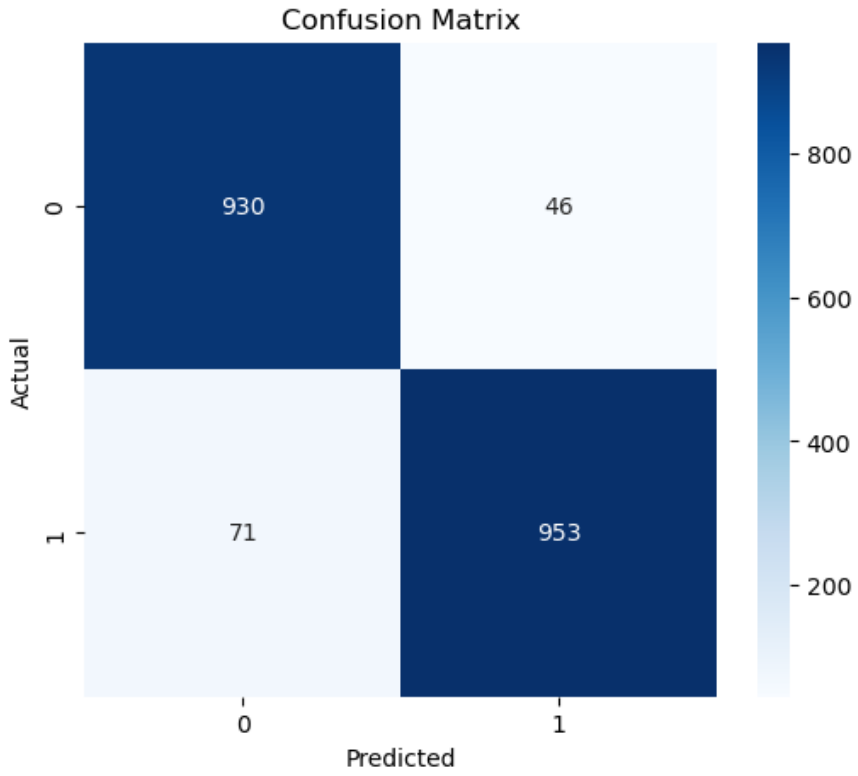
16.1 Confusion Matrix

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, rf_pred)

cm

array([[930,  46],
       [ 71, 953]])
```



Original Random Forest confusion-matrix visualization from the notebook.

True Positive: the model predicts refill and the actual class is refill.

True Negative: the model predicts no refill and the actual class is no refill.

False Positive: the model predicts refill when the actual class is no refill.

False Negative: the model predicts no refill when the actual class is refill.

For refill planning, recall is especially relevant because false negatives correspond to cases where an actual refill requirement is missed. This is an interpretation of the metric's operational meaning, not a claim that the notebook optimized for recall.

16.2 Classification Report

```
from sklearn.metrics import classification_report

print(classification_report(y_test, rf_pred))
```

```
precision    recall  f1-score   support

           0       0.93      0.95      0.94       976
           1       0.95      0.93      0.94      1024

    accuracy                           0.94      2000
   macro avg       0.94      0.94      0.94      2000
weighted avg       0.94      0.94      0.94      2000
```

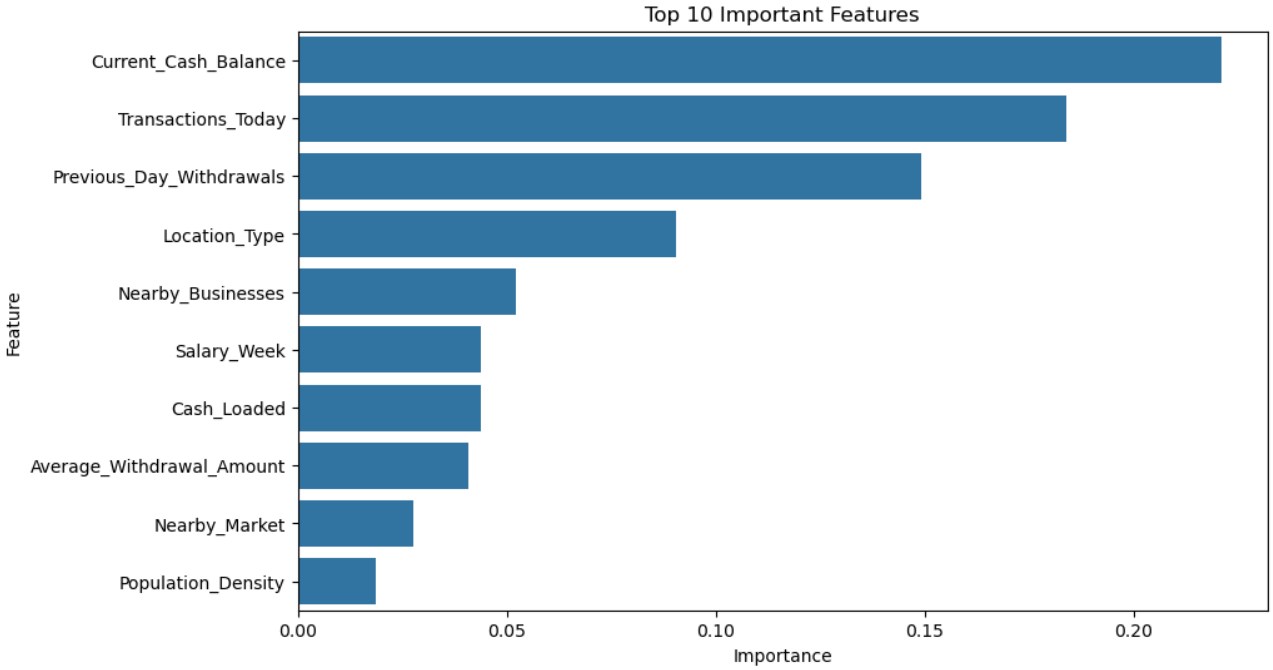
17. Feature Importance

The notebook calculates feature_importances_ from the original Random Forest model. Feature importance indicates how much the fitted tree ensemble used each feature for its splits; it should not be interpreted as proof of causation.

The highest-ranked features in the supplied notebook are Current_Cash_Balance, Transactions_Today, Previous_Day_Withdrawals, Location_Type, and Nearby_Businesses.

Feature	Importance
Current_Cash_Balance	0.220939

Feature	Importance
Transactions_Today	0.183924
Previous_Day_Withdrawals	0.149238
Location_Type	0.090397
Nearby_Businesses	0.052213
Salary_Week	0.043912
Cash_Loaded	0.043828
Average_Withdrawal_Amount	0.040755
Nearby_Market	0.027677
Population_Density	0.018627
Nearest_Bank_Distance_km	0.016135
ATM_ID	0.015791
Date	0.015626
Temperature_C	0.015624
Festival_Season	0.013246
ATM_Age_Years	0.012167
Month	0.009459
Day_of_Week	0.008226
Holiday	0.006871
Working_Hours	0.004571
Weather	0.004327
ATM_Type	0.002484
Weekend	0.002282
Maintenance_Due	0.001681



Original top-10 feature-importance visualization from the notebook.

18. Hyperparameter Tuning

The notebook uses GridSearchCV to evaluate combinations of Random Forest hyperparameters with 5-fold cross-validation and accuracy as the scoring metric.

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}

grid_search = GridSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)
```

Parameter grid used:

Parameter	Values tested
n_estimators	[100, 200]
max_depth	[None, 10, 20]
min_samples_split	[2, 5]
min_samples_leaf	[1, 2]

Best parameters reported by the notebook:

```
Best Parameters:
{'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}

Best Accuracy:
0.953375
```

The best cross-validation accuracy reported by GridSearchCV is 0.953375. This is a cross-validation score, not the final held-out test accuracy.

19. Final Optimized Model

The notebook selects grid_search.best_estimator_, predicts the held-out test set, and reports the final test accuracy.

```
best_model = grid_search.best_estimator_

best_predictions = best_model.predict(X_test)

print("Test Accuracy:", accuracy_score(y_test, best_predictions))

Test Accuracy: 0.9415
```

Final tuned Random Forest test accuracy: 0.9415 (94.15%).

Important distinction: the notebook's overall highest direct model test accuracy is Gradient Boosting at 96.45%, while the tuned Random Forest reaches 94.15% on the held-out test set. The GridSearchCV result of 95.3375% is its best 5-fold cross-validation score.

20. Model Saving and Loading

The notebook uses Joblib to persist the selected Random Forest model and then loads it back to demonstrate reuse.

Saving the model

```
import joblib
```

```
joblib.dump(best_model, "best_model.pkl")
```

Loading the model

```
loaded_model = joblib.load("best_model.pkl")
```

Making predictions

```
loaded_model.predict(X_test.iloc[:5])
```

```
array([0, 1, 0, 0, 0])
```

The notebook therefore demonstrates a reusable model artifact named **best_model.pkl** and a prediction call on the first five test rows.

21. End-to-End Machine Learning Workflow

01	**Raw ATM Data**
02	**Data Understanding**
03	**Data Cleaning / Checks**
04	**Exploratory Data Analysis**
05	**Feature & Target Selection**
06	**Train-Test Split**
07	**Model Training**
08	**Model Comparison**
09	**Hyperparameter Tuning**
10	**Final Model**
11	**Prediction**
12	**ATM Refill Decision**

The workflow above reflects the sequence implemented in the supplied notebook, with the business decision shown as the potential downstream use of the prediction.

22. Business Interpretation

The following are potential business applications of a refill-prediction model; they were not implemented as operational systems in the supplied notebook.

- Identify ATMs that may require cash replenishment.
- Reduce the risk of ATM cash-outs by prioritizing potentially high-risk machines.
- Support refill planning using predicted requirements.
- Help operations teams make more data-informed decisions.
- Potentially reduce unnecessary refill trips when predictions are reliable.
- Support customer service by improving the planning of cash availability.

23. Project Results

Metric	Notebook result
Dataset size	10,000 rows × 25 columns
Model features	24
Target	Needs_Refill
Models tested	Logistic Regression, Decision Tree, Random Forest, Gradient Boosting
Best direct test accuracy	Gradient Boosting — 0.9645
Random Forest test accuracy	0.9415
GridSearchCV best CV accuracy	0.953375
Best GridSearchCV parameters	max_depth=None; min_samples_leaf=1; min_samples_split=2; n_estimators=100
Top Random Forest feature	Current_Cash_Balance — 0.220939
Model persistence	Joblib → best_model.pkl

24. Limitations

These are realistic limitations or considerations for a portfolio ML system. They are not claims that the notebook already solves them.

• The model depends on the historical data represented in the notebook.

• Model quality depends on data quality and the representativeness of future observations.

• Real-world ATM demand can change over time.

• External factors may influence cash requirements beyond the variables present in this dataset.

• The notebook does not demonstrate real-time data ingestion or production monitoring.

• The preprocessing uses LabelEncoder on categorical columns, including the target; a production pipeline should explicitly preserve and version the exact encoding mappings.

25. Future Improvements

• Integrate real-time or regularly refreshed ATM transaction data.

• Evaluate time-series forecasting approaches for demand-aware refill planning.

• Add richer feature engineering around transaction trends, cash depletion rates, and temporal patterns.

• Explore cost-sensitive classification so missed refill requirements can be weighted appropriately.

• Add model monitoring and drift detection.

• Build an automated prediction pipeline.

• Expose predictions through an API.

• Connect predictions to a Power BI or similar operational dashboard.

• Deploy the model and preprocessing pipeline in a cloud environment.

26. Recruiter-Friendly Project Summary

How I Would Explain This Project in an Interview

“This project is a machine learning system for predicting whether an ATM needs a cash refill. The dataset contains 10,000 ATM observations with 25 columns covering transaction, cash-balance, location, calendar, and operational information. I cleaned and inspected the data, encoded categorical features, selected Needs_Refill as the target, and split the data into 80% training and 20% testing sets. I compared Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting. Gradient Boosting achieved the highest test accuracy at 96.45%. I also evaluated Random Forest using a confusion matrix, classification report, and feature importance, then used GridSearchCV to tune its hyperparameters. Finally, I saved the selected model using Joblib so it can be loaded and reused for future predictions. The business goal is to support better ATM refill planning and reduce the risk of cash shortages.”

27. Important Interview Questions

1. Why is this a classification problem?

Because the target Needs_Refill represents a class label rather than a continuous numerical value.

2. Why did you choose Needs_Refill as the target?

It is the business outcome the model is designed to predict: whether an ATM needs replenishment.

3. Why did you use a train-test split?

To train models on one portion of the data and evaluate them on unseen observations.

4. Why is test_size=0.2 used?

It reserves 20% of the observations for testing and leaves 80% for training.

5. Why random_state=42?

It makes the train-test split reproducible.

6. Why did you use Logistic Regression?

It provides a simple classification baseline for comparison.

7. Why did you use Random Forest?

It is an ensemble of decision trees and can capture non-linear relationships.

8. Why did Gradient Boosting perform better in your comparison?

In this notebook, it achieved the highest measured test accuracy, 0.9645. The notebook does not establish a causal reason for that result.

9. What is GridSearchCV?

It systematically tests the specified hyperparameter combinations using cross-validation and selects the combination with the best cross-validation score.

10. Why did you tune Random Forest?

To search for a stronger Random Forest configuration rather than relying only on its initial settings.

11. What does n_estimators mean?

The number of trees used by the Random Forest.

12. What does max_depth mean?

The maximum depth allowed for each decision tree.

13. What is a confusion matrix?

A table showing predicted versus actual classes, including true positives, true negatives, false positives, and false negatives.

14. Why can recall be important here?

A false negative means an actual refill requirement was predicted as no refill, so recall helps assess how well refill cases are being captured.

15. What is feature importance?

A model-derived measure showing which features contributed more to the Random Forest's split decisions; it is not proof of causation.

16. What did your Random Forest confusion matrix show?

It was [[930, 46], [71, 953]] on the 2,000-row test set.

17. What was the Random Forest classification report?

For class 0: precision 0.93, recall 0.95, F1 0.94; for class 1: precision 0.95, recall 0.93, F1 0.94; overall accuracy 0.94.

18. How did you prevent data leakage?

The notebook separates the target from the features before the train-test split. However, it does not implement a formal sklearn Pipeline, so a production version should make preprocessing and training boundaries explicit.

19. How would you deploy this model?

Package the preprocessing and trained model together, expose a prediction interface such as an API, and connect it to a monitored data pipeline.

20. How would you improve the project?

Add temporal feature engineering, real-time data, cost-sensitive evaluation, monitoring, automated pipelines, dashboard/API integration, and cloud deployment.

28. Code Quality Review

• The supplied notebook uses clear variable names such as X_train, y_train, rf_model, grid_search, and feature_importance.

• Major model steps are separated into individual cells.

• The notebook includes model evaluation, tuning, and persistence rather than stopping at model training.

• For a production-ready notebook, repeated imports could be consolidated and the preprocessing/model steps could be placed into a single reproducible Pipeline.

• The final PDF preserves the actual methodology and measured outputs rather than replacing them with invented results.

29. Visual / Portfolio Presentation Notes

• The document uses consistent section headings, tables, code blocks, charts, whitespace, and page numbering.

• Original notebook visualizations are reused where available.

• The local dataset path is not presented as a professional deployment path.

• Business applications and future improvements are clearly distinguished from completed notebook functionality.

30. Final Quality Check

Check	Status
Important notebook sections represented	Yes
Existing code preserved where relevant	Yes
Existing outputs/results used	Yes
Fabricated accuracy or dataset values	None intentionally introduced
Logistic Regression included	Yes
Decision Tree included	Yes
Random Forest included	Yes
Gradient Boosting included	Yes
Random Forest + GridSearchCV included	Yes
Feature importance included	Yes
Model saving/loading included	Yes
Charts included from notebook outputs	Yes, where PNG outputs were available
Recruiter-friendly interview explanation	Yes
Interview questions and concise answers	Yes

Source note

This PDF was created from the supplied Smart ATM Cash Refill Prediction System Jupyter Notebook and the accompanying project-generation instructions. No external dataset or web source was used to fill missing notebook results.